

UNIVERSIDAD TECNOLÓGICA METROPOLITANA
INFB6074 — Infraestructura para C. de Datos

Segunda Evaluación Integral

CrediRapido: Infraestructura Reproducible, Orquestación Asíncrona
y Registro Verificable para un Motor de Scoring Crediticio

Integrante	
Email	
Modalidad	Individual (1 estudiante)
Ejercicios desarrollados	Ejercicio 2, Ejercicio 3, Ejercicio 4
Curso	INFB6074 — Ingeniería Civil en Ciencia de Datos
Docente	Dr. Ing. Michael Miranda Sandoval
Fecha de entrega	

b. Resumen ejecutivo

Este informe documenta el diseño, implementación y validación de **CrediRapido**, un prototipo de motor de scoring de solicitudes de crédito de consumo construido para resolver tres problemas de infraestructura de ciencia de datos: (1) desacoplar una tarea de cómputo con múltiples pasos y latencia de la interfaz que la solicita, (2) hacerlo en un ambiente reproducible y auditable, y (3) dejar evidencia verificable de cada decisión tomada por el sistema. La solución combina una interfaz Streamlit, una cola Redis, dos Workers Python y una cadena de bloques didáctica que audita el proceso completo.

Los tres ejercicios obligatorios para la modalidad individual (Ejercicio 2: ambiente Docker; Ejercicio 3: orquestador asíncrono; Ejercicio 4: registro verificable) fueron implementados y ejecutados de punta a punta con evidencia real: 4 solicitudes procesadas por 2 Workers concurrentes, 26 eventos de auditoría generados y una cadena de 27 bloques (génesis + 26) validada, alterada intencionalmente y revalidada, detectando la alteración exactamente en el bloque afectado. La recomendación final es que, para este escenario, un registro tipo blockchain aporta valor solo como capa de integridad sobre decisiones de alto impacto (aprobación/rechazo de crédito), y no como reemplazo de la base de datos operacional.

c. Contexto conceptual

El proyecto articula los contenidos trabajados desde la clase 5: procesamiento e ingesta de solicitudes, orquestación de flujos con estado, administración y calidad de datos de entrada, Docker como mecanismo de reproducibilidad, arquitecturas asíncronas con interfaz-cola-Worker, Redis como coordinador de tareas y almacén temporal de estado, y fundamentos de blockchain (hash determinista, bloques enlazados, validación, alteración controlada, prueba de trabajo didáctica y árboles de Merkle) como mecanismo de registro verificable.

A diferencia de la primera evaluación integradora, aquí el foco no está en benchmarks de hardware ni en el formato de almacenamiento de archivos, sino en el *comportamiento del sistema en el tiempo*: cómo se mueve una solicitud por distintos estados, qué pasa cuando un Worker falla o cuando los datos de entrada son inválidos, y cómo se puede demostrar, después del hecho, que el historial de decisiones no fue alterado.

d. Escenario aplicado

CrediRapido es una fintech ficticia que evalúa solicitudes de crédito de consumo. Cada solicitud pasa por seis pasos: validación de formato, verificación de reglas duras de elegibilidad, cálculo de score financiero (relación deuda/ingreso y monto/ingreso), cálculo de score de historial y antigüedad laboral, consulta simulada a un buró de crédito externo (con latencia artificial) y cálculo de la decisión final (aprobado / revisión manual / rechazado). Esta secuencia de pasos, con latencia acumulada de varios segundos, es exactamente el tipo de tarea que **no conviene ejecutar dentro de la interfaz**: bloquearía la UI, impediría atender más de una solicitud a la vez y no dejaría rastro de *cómo* se llegó a la decisión, algo crítico en un dominio regulado como el crédito.

Datos de entrada

nombre_solicitante, monto_solicitado, ingreso_mensual, deuda_mensual_actual, antigüedad_laboral_meses, edad, historial_credicio (bueno/regular/malo), region. Validaciones

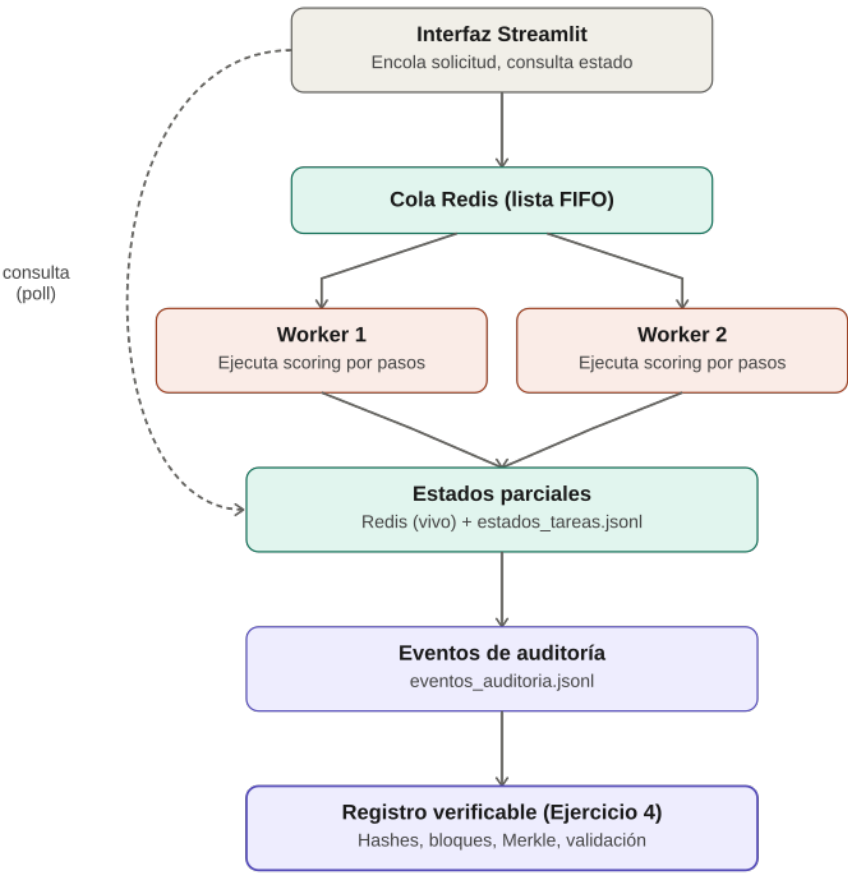
iniciales: tipos numéricos correctos, presencia de todos los campos y pertenencia del historial crediticio a las categorías válidas (ver estado.py).

Salida esperada

Un resultado con score_final (0-100), decisión (aprobado / revision_manual / rechazado) y los sub-scores que la componen, junto con el registro completo de auditoría del proceso.

e. Arquitectura del flujo

La interfaz encola la solicitud y solo consulta el estado publicado por el Worker (nunca ejecuta el scoring). Redis actúa como broker (lista FIFO, BLP0P) y como almacén temporal del último estado conocido de cada tarea. Cada Worker ejecuta los 6 pasos, publicando avance en Redis y registrando eventos persistentes de auditoría.



Gris = interfaz · Teal = coordinación (Redis) · Coral = Workers · Púrpura = auditoría / registro verificable

Tabla de estados y responsables

Estado	Quién escribe	Quién lee
recibido	Interfaz (al encolar)	Worker (al tomar la tarea)

Estado	Quién escribe	Quién lee
procesando	Worker (pasos 1-4)	Interfaz (polling)
publicando_avance	Worker (paso 5, consulta externa)	Interfaz (polling)
completado / fallido	Worker (paso 6 o manejo de error)	Interfaz, script de auditoría

Eventos de auditoría definidos (mínimo 6 exigidos)

- tarea_creada — al encolar desde la interfaz.
- tarea_iniciada — cuando un Worker toma la tarea de la cola.
- avance_parcial — uno por cada paso del scoring (5 por tarea exitosa).
- error — excepción de validación, procesamiento o timeout.
- tarea_completada — al publicar el resultado final.
- verificacion — chequeo de consistencia posterior a completar la tarea.

f. Ambiente reproducible (Ejercicio 2)

El ambiente se define mediante `docker/Dockerfile` (imagen base fija `python:3.11.9-slim`, dependencias en `requirements.txt`) y `docker/docker-compose.yml`, que orquesta 4 servicios: `redis` (imagen `redis:7.2-alpine`, con `healthcheck`), `worker1` y `worker2` (dos instancias del mismo `Worker` para observar reparto de carga) y `interfaz` (`Streamlit`, puerto 8501). Variables de entorno declaradas en `.env.example` sin credenciales reales. La carpeta `outputs/` se monta como volumen compartido entre `host` y contenedores para persistir estados, eventos y la cadena de auditoría fuera del ciclo de vida de los contenedores.

Comandos: `docker compose up --build`, `docker compose ps`, `docker compose logs worker1`, `docker compose down -v` (ver `README.md` para el detalle completo).

Transparencia sobre el entorno de desarrollo: el sandbox de trabajo usado para construir y probar este repositorio no tenía Docker Engine disponible. Por esa razón, la validación de punta a punta (Ejercicio 3 y 4) se ejecutó con Redis nativo instalado vía `apt-get`, replicando exactamente la misma lógica que corre dentro de los contenedores (mismo código, misma configuración de red lógica vía variables `REDIS_HOST/REDIS_PORT`). Los archivos Docker siguen la especificación exacta de la pauta y las buenas prácticas revisadas en clase, pero se recomienda ejecutar `docker compose up --build` en un equipo con Docker Desktop/Engine para generar la captura de pantalla de servicios corriendo en contenedores, como evidencia adicional para la defensa.

Qué hace reproducible al ambiente / qué sigue dependiendo del host

Reproducible por Docker	Depende del host
Versión exacta de Python (3.11.9) y Redis (7.2-alpine)	Versión de Docker Engine / Docker Compose instalada
Dependencias Python fijadas (<code>requirements.txt</code> con versión exacta)	Sistema operativo (Linux nativo vs. WSL2 en Windows vs. macOS)
Red interna aislada entre servicios (<code>credirapido_net</code>)	Recursos disponibles (CPU, RAM, I/O de disco del host)
Variables de entorno declaradas explícitamente	Latencia de red host-contenedor en configuraciones WSL2

g. Orquestación asíncrona (Ejercicio 3)

Se probó el flujo completo encolando 4 solicitudes (una aprobación, un rechazo por score, un rechazo automático por regla dura de edad, y una solicitud con dato inválido) con 2 Workers corriendo en paralelo. El reparto de trabajo vía `BLPOP` se observó directamente en los logs: cada Worker tomó tareas distintas sin coordinación explícita adicional, demostrando que Redis reparte la carga de forma nativa entre consumidores disponibles.

task_id	Solicitante	Worker asignado	Resultado	Score final
bb8158a4	Ana Soto	worker-vm-757	aprobado	82.15
218b978a	Luis Vega	worker-vm-758	rechazado	28.48
ba9ab2ce	Menor Edad	worker-vm-757	rechazado (regla dura: edad)	0
ade876f6	Dato Invalido	worker-vm-758	fallido (SolicitudInvalida)	—

Manejo de errores implementado

- **Tarea inválida:** campo no numérico (`monto_solicitado='no-es-numero'`) detectado en el paso 1 (`validar_formato`), estado final 'fallido', sin propagar excepción sin control.
- **Excepción durante procesamiento:** capturada de forma genérica en `worker.py` (bloque `try/except Exception`), registrando el tipo y detalle del error como evento de auditoría 'error'.
- **Timeout simulado:** cada paso verifica el tiempo transcurrido contra `TASK_TIMEOUT_SECONDS` (8s por defecto); si se excede, se lanza `TimeoutError` y la tarea se marca 'fallido' con el motivo explícito.

Qué ocurre en escenarios límite (criterio de análisis)

Si la interfaz se cierra: no afecta al procesamiento — la tarea ya fue encolada y el Worker la completa igual; el resultado queda disponible en Redis (con TTL de 1 hora) y en el log persistente, listo para ser consultado cuando la interfaz vuelva a abrirse. *Si un Worker falla* a mitad de una tarea (crash del proceso): la tarea ya fue retirada de la cola (BLPOP la extrae de forma atómica), por lo que se pierde — ningún otro Worker la retomará automáticamente. Esto se identifica como riesgo explícito en la sección de riesgos. *Si hay más tareas que Workers:* las tareas excedentes esperan en la cola FIFO hasta que un Worker quede libre; no hay pérdida de datos, solo latencia adicional. *Si se levantan dos Workers para la misma cola* (como en esta prueba): Redis garantiza que cada mensaje es entregado a un solo consumidor vía BLPOP atómico, logrando reparto de carga sin necesidad de lógica adicional de coordinación.

h. Registro verificable (Ejercicio 4)

Se implementó una cadena de bloques didáctica (`blockchain_audit/blockchain.py`) sobre los 26 eventos de auditoría reales generados por el Ejercicio 3, resultando en 27 bloques (génesis + 26). Cada bloque incluye índice, timestamp, datos del evento, hash del bloque anterior, nonce y hash SHA-256 propio, calculado sobre una serialización estable (`json.dumps(..., sort_keys=True)`).

Validación y alteración controlada

La cadena recién construida validó correctamente (`validar_cadena() -> True`). Se alteró intencionalmente el bloque #13 (evento de tipo `avance_parcial` de la tarea 218b978a) reemplazando su contenido sin recalcular el hash, simulando un intento de manipulación directa del registro. La revalidación detectó la inconsistencia exactamente en ese bloque:

```
hash_almacenado = 00b440088baf874877eee220328924dcfd5e87d07a667daa32c17aae1a84df8f
hash_recalculado = ee60cf922a88740b423ab8db0b3918226783edbc2d61e2cd88f2ccfa6bd09e8b
primer_bloque_invalido = 13
```

Esto confirma la propiedad central del ejercicio: cualquier alteración posterior a la escritura queda matemáticamente detectable, sin depender de un tercero de confianza.

Árbol de Merkle por tarea

Para la tarea bb8158a4 (Ana Soto, aprobada), se agruparon sus 9 eventos de auditoría en una raíz de Merkle única: b501029cefd9f4811bce570c78a7dc71972fecfcaf63b38375066485e27db045. Esta raíz resume todos los eventos de la tarea en un solo hash: si cualquiera de los 9 eventos cambia, la raíz cambia, permitiendo verificar la integridad de la tarea completa sin recorrer cada evento individualmente.

Benchmark de Prueba de Trabajo (PoW) didáctica

Dificultad (prefijo)	Intentos	Tiempo (s)
0	15	0.0003
00	155	0.0013
000	2 995	0.0226

El crecimiento es consistente con lo esperado teóricamente: cada carácter '0' adicional en el prefijo reduce el espacio de hashes válidos en un factor ~16 (base hexadecimal), por lo que el número de intentos promedio también crece aproximadamente 16x por nivel de dificultad (15 -> 155 -> 2 995 es coherente con esa progresión, con la variabilidad propia de un proceso estocástico).

Por qué este prototipo no es una blockchain productiva: no existe red P2P ni múltiples nodos independientes verificando la cadena; no hay consenso distribuido (ej. el nodo que genera los bloques también los valida); no hay incentivos económicos que hagan costoso un ataque; y la dificultad de PoW usada (2-3 ceros) es puramente didáctica, muy por debajo de lo necesario para resistir cómputo real. La garantía que sí aporta es evidencia de integridad local y de bajo costo operativo, útil para auditoría interna.

i. Trazabilidad integral

Tabla que relaciona tarea, estado, timestamp, Worker, evento, hash del evento (bloque) para la tarea bb8158a4 (Ana Soto), reconstruyendo su ciclo de vida completo desde creación hasta verificación:

Bloque	Evento	Worker	Hash (primeros 16 car.)
#1	tarea_iniciada	worker-vm-757	0067866b04189b39...
#2	tarea_creada	interfaz	00a66c79ee61b591...
#7	avance_parcial (10%)	worker-vm-757	00ae05aba829d7ab...
#9	avance_parcial (25%)	worker-vm-757	00da7cf32a2a6df5...
#11	avance_parcial (45%)	worker-vm-757	00d706d568139b8c...
#14	avance_parcial (65%)	worker-vm-757	009a82a346f3c233...
#19	avance_parcial (85%)	worker-vm-757	0065be440569c93d...
#20	tarea_completada	worker-vm-757	00e9fe4a0597e01e...
#21	verificacion	worker-vm-757	00f34dc5b99e360c...

Nota: los índices de bloque no son consecutivos para una misma tarea porque la cadena se construye en el orden real de llegada de eventos entre las 4 tareas procesadas concurrentemente por 2 Workers — esto es evidencia adicional de que el registro captura el orden real de ejecución, no un orden artificial agrupado por tarea.

j. Evaluación arquitectónica: log / base de datos / blockchain

Criterio	Archivo log	Base de datos	Registro tipo blockchain
Facilidad de implementación	Muy alta	Alta	Media
Costo operativo	Muy bajo	Medio (requiere motor de BD)	Bajo-medio (CPU de minado)
Consultas estructuradas	Baja (requiere parseo)	Alta (SQL, índices)	Baja (requiere recorrer cadena)
Evidencia de no-alteración	Ninguna (editable sin rastro)	Ninguna (UPDATE sin rastro)	Alta (detecta alteración exacta)
Escalabilidad de escritura	Alta	Alta	Media (minado agrega latencia)
Adecuado como fuente operacional	No	Sí	No (es complemento, no reemplazo)

Recomendación para el escenario de CrediRapido: usar **base de datos** como fuente operacional (consultas, reportes, dashboard de solicitudes) y **registro tipo blockchain** como capa adicional específicamente para los eventos que representan la decisión final (aprobado/rechazado) y su cadena de cálculo — es decir, no para todo el sistema, sino para el subconjunto de eventos donde la integridad demostrable importa más que la velocidad de consulta. El archivo log (JSONL) se mantiene como bitácora de bajo costo y respaldo legible por humanos, pero no sustituye a ninguno de los otros dos. Blockchain no se presenta como solución universal: su costo (latencia de minado, dificultad de hacer consultas ad-hoc) solo se justifica donde la propiedad de detección de alteración aporta valor real.

k. Riesgos y limitaciones

- **Pérdida de Worker a mitad de tarea:** BLPOP retira la tarea de la cola de forma atómica; si el Worker muere después de eso, la tarea se pierde sin reintento automático. Mitigación propuesta: patrón de cola de reintentos (requeue tras timeout de heartbeat).

- **Duplicidad de tareas:** si la interfaz reenvía la misma solicitud por un doble clic, se generan dos `task_id` distintos para la misma solicitud real. Mitigación: idempotencia por hash del payload antes de encolar.
- **Mensajes mal formados:** cubierto parcialmente por `validar_formato()`, pero un payload malformado a nivel JSON (no parseable) haría fallar el Worker antes de llegar a esa validación; falta un `try/except` adicional alrededor del parseo inicial en `obtener_siguiente_tarea()`.
- **Crecimiento de logs:** `estados_tareas.jsonl` y `eventos_auditoria.jsonl` crecen sin límite; en producción requerirían rotación y archivado periódico.
- **Exposición de datos sensibles:** los payloads (ingreso, deuda, edad) viajan y se almacenan sin cifrar en Redis ni en los JSONL; en un escenario real esto exige cifrado en tránsito y reposo, y control de acceso a los volúmenes.
- **Falsa sensación de inmutabilidad:** la cadena de bloques es generada y almacenada por el mismo proceso que la audita; un atacante con acceso al sistema podría regenerar la cadena completa desde cero. La garantía es de integridad detectable, no de inmutabilidad absoluta.
- **Complejidad operativa:** operar Redis + N Workers + minado de bloques agrega piezas móviles frente a una arquitectura síncrona simple; se justifica solo porque la tarea de negocio (scoring) efectivamente requiere desacoplamiento.

I. Conclusiones

La implementación demuestra, con evidencia de ejecución real (no solo diseño en papel), que una arquitectura interfaz-cola-Worker desacopla efectivamente el cómputo costoso de la experiencia de usuario, que Redis reparte carga entre Workers sin coordinación adicional, y que un registro tipo blockchain detecta alteraciones puntuales en un log de auditoría de forma verificable. El aprendizaje central es que ninguna de estas piezas de infraestructura (Docker, Redis, blockchain) es valiosa por sí misma: cada una resuelve un problema específico de este escenario (reproducibilidad, desacoplamiento, integridad demostrable) y su costo de complejidad debe justificarse contra ese problema puntual, no asumirse como buena práctica universal.

Como mejora futura, se identifican tres líneas: (1) implementar reintentos automáticos ante caída de Worker (idealmente con un patrón de cola de reintentos con backoff), (2) cifrar los datos sensibles del payload antes de persistirlos, y (3) evaluar Redis Streams si en el futuro se requiere que múltiples sistemas (no solo Workers) consuman el mismo evento de forma independiente.

m. Referencias y anexos

Ver `referencias/referencias.md` para la lista completa de fuentes. Evidencia adicional (logs de ejecución, JSON completo de la cadena, capturas) disponible en `outputs/` y `outputs/evidencias/`: `estados_tareas.jsonl`, `eventos_auditoria.jsonl`, `auditoria_cadena.json`, `merkle_por_tarea.json`, `benchmark_pow.csv`, `worker1.log`, `worker2.log`.

Declaración de uso de IA: se utilizó Claude (Anthropic) como asistente de desarrollo para estructurar el proyecto, redactar código base y este informe. Todo el código fue ejecutado y validado en un entorno real con Redis (no simulado), con revisión y adaptación de las decisiones de diseño (escenario,

justificaciones técnicas, análisis de riesgos). Ver README.md para el detalle completo de esta declaración.